

7. LECTURA Y ESCRITURA DE INFORMACIÓN EN FICHEROS.....	1
7.1. Lectura de información binaria en ficheros.....	1
7.2. Lectura de información en ficheros de texto.....	3
7.2.1. FileReader y BufferedReader.....	3
7.2.2. Scanner.....	4

7. LECTURA Y ESCRITURA DE INFORMACIÓN EN FICHEROS



7.1. Lectura de información binaria en ficheros

Para leer el contenido de un fichero en formato binario, es decir, byte a byte, utilizaremos la clase `java.io.FileInputStream`, clase hija de `java.io.InputStream`.

Para crear un objeto de lectura de fichero por bytes tendremos que pasar como parámetro la ruta del fichero que deseamos leer o un objeto `File` que haga referencia a dicho fichero.

```
public FileInputStream(File file) throws FileNotFoundException
```

```
public FileInputStream(String name) throws FileNotFoundException
```

```
FileInputStream lector1 = new FileInputStream("./archivo02.txt");  
File archivo03 = new File("./archivo03.txt");  
FileInputStream lector2 = new FileInputStream(archivo03);
```

Para leer los datos de este objeto tenemos dos opciones principales, leer carácter a carácter o en bloques de n caracteres.

```
public int read() throws IOException
```

```
public int read(byte[] b) throws IOException
```

Hay que destacar que la primera opción nos devuelve un valor entero, correspondiente al byte que ha leído, siendo -1 en el caso de no leer nada, mientras que a la segunda opción pasamos por parámetro un array de bytes que se rellenará, total o parcialmente, con el contenido leído, devolviendo el número de bytes que ha leído. El resultado será -1 si no ha leído nada.

Veamos un ejemplo de cada uno:

```
int palabra;
do{
    palabra = lector2.read();
    if (palabra == -1)
        break;
    else{
        System.out.print(palabra + " ");
    }
}while(true);
Lector2.close();
```

```
byte[] lectura = new byte[100];
int longitud;

do{
    longitud = lector1.read(lectura);
    if (longitud == -1)
        break;
    else{
        for(int i=0; i<longitud; i++){
            System.out.print(lectura[i] + " ");
        }
    }
    System.out.println("");
}while(true);
lector1.close();
```

Algo muy importante, que no debemos olvidar, es cerrar el fichero después de terminar de leer y/o escribir en él, ya que, de lo contrario, puede quedarse bloqueado y no podremos acceder a su contenido.

Para ello vamos a utilizar el método `close()`.

7.2. Lectura de información en ficheros de texto

Para este tipo de lectura contamos con dos opciones.

7.2.1. FileReader y BufferedReader

En primer lugar tenemos la clase `java.io.FileReader`, la cual podemos combinar con `java.io.BufferedReader` para leer línea a línea en lugar de carácter a carácter.

Para crear un objeto `FileReader` podemos pasar por parámetro un objeto `File` o el nombre del fichero que queremos leer.

```
public FileReader(File file) throws FileNotFoundException
```

```
public FileReader(String fileName) throws FileNotFoundException
```

Y a la hora de crear el objeto `BufferedReader` pasaremos por parámetro el objeto `FileReader` que hayamos creado previamente.

```
public BufferedReader(Reader in)
```

Ejemplos:

```
File archivo = new File("archivo.txt");  
FileReader lector1 = new FileReader(archivo);  
BufferedReader lectorBuff1 = new BufferedReader(lector1);
```

```
FileReader lector2 = new FileReader("archivo.txt");  
BufferedReader lectorBuff2 = new BufferedReader(lector2);
```

El objeto `FileReader`, de por sí, ya nos permitirá leer el contenido del fichero, pero principalmente carácter a carácter, mediante el método `read()`, que devuelve un entero correspondiente al valor del carácter.

Mientras que si utilizamos el objeto `BufferedReader` podremos realizar también la lectura carácter a carácter, con `read()`, o línea a línea con `readLine()`. Este último método nos devolverá el `String` correspondiente a la línea que ha leído, con un resultado de `null` en el caso de llegar al final del fichero.

Lectura carácter a carácter:

```
int dato;
do{
    dato = lector1.read();
    if(dato == -1)
        break;
    else
        System.out.print(dato + " ");
}while(true);
lector1.close();
System.out.println("");
```

Lectura línea a línea:

```
String linea;
do{
    linea = lectorBuff2.readLine();
    if(linea == null)
        break;
    else
        System.out.println(linea);
}while(true);
lectorBuff2.close();
System.out.println("");
```

7.2.2. Scanner

La segunda opción para realizar la lectura del contenido de un fichero de texto será utilizando la clase Scanner pasándole un objeto File por parámetro, con el cual indicaremos de qué fichero queremos leer.

En este caso solo podremos utilizar los métodos `next()` y `nextLine()` para recoger línea a línea los datos del fichero. También haremos uso de los métodos `hasNext()` y `hasNextLine()` para realizar una comprobación previa de si queda alguna línea por leer.

```
String datos;  
  
File ficheroDatos = new File("archivo03.txt");  
Scanner lectorDatos = new Scanner(ficheroDatos);  
while (lectorDatos.hasNextLine()) {  
    datos = lectorDatos.nextLine();  
    System.out.println(datos);  
}  
  
lectorDatos.close();
```

Links a la ayuda de java de todas las clases que hemos visto para la lectura de ficheros:

<https://docs.oracle.com/javase/8/docs/api/java/io/FileInputStream.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/FileReader.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/BufferedReader.html>

<https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html>

Pongámonos en acción con la lectura de ficheros:

